Entwicklungsdokumentation: Pablo - Desk-Companion mit Sprachinteraktion

Helena Bongartz, Elias Bürgin, Paul Siewert

3. April 2026

Inhaltsverzeichnis

1 Einleitung **2**

2 Entwicklungsprozess **3**
2.1 Konzeption . . . 3
2.2 Iterative Entwicklung . . . 3

3 Darstellung der Ergebnisse **4**
3.0.1 Funktionale Anforderungen . . . 4
3.0.2 Nicht-funktionale Anforderungen . . . 5
3.1 Softwarearchitektur . . . 5
3.1.1 (1) Aktivierung des Systems . . . 6
3.1.2 (2) Spracherkennung (STT) . . . 6
3.1.3 (3) LLM-Verarbeitung . . . 6
3.1.4 (4) Sprachausgabe (TTS) . . . 6
3.2 Hardwarekonzept . . . 6

4 Diskussion der Ergebnisse **7**
4.1 Physischer Companion statt reiner Softwareassistent . . . 8
4.2 Modular erweiterbare Plugin-Struktur . . . 9
4.3 Raspberry Pi als zentrale Recheneinheit . . . 9
4.4 Wakeword- und Sprachsteuerung . . . 9
4.5 Visuelle Persona . . . 9
4.6 Kompaktes integriertes Display . . . 10

5 Praktische Herausforderungen **10**

6 Ergebnis und kritische Einordnung **10**
6.1 Praktische Herausforderungen . . . 11
6.2 Ausblick . . . 12

1 Einleitung

Der Umgang mit digitalen Assistenzsystemen hat sich in den vergangenen Jahren grundlegend verändert. Sprachbasierte Systeme wie Amazon Alexa, Google Assistant und Apple Siri sind heute in Millionen von Haushalten verfügbar und ermöglichen eine freihändige, niedrigschwellige Interaktion im Alltag [1], [2]. Parallel dazu haben Fortschritte im Bereich der großen Sprachmodelle (Large Language Models, LLMs) die Qualität maschinell generierter Antworten erheblich gesteigert, sodass natürlichsprachliche Dialoge zunehmend flüssig und kontextbezogen geführt werden können [3]. Diese Entwicklung hat die Erwartungen an digitale Assistenzsysteme nachhaltig verändert: Nutzende setzen zunehmend voraus, dass Systeme nicht nur einzelne Befehle ausführen, sondern flexibel auf offene Fragestellungen reagieren und sich in individuelle Arbeitsabläufe einfügen.

Trotz der breiten Verbreitung sprachbasierter Assistenten bestehen wesentliche Einschränkungen, die deren Einsatz im persönlichen Umfeld begrenzen. Kommerzielle Systeme sind überwiegend als geschlossene Plattformen konzipiert, deren Funktionsumfang herstellerseitig definiert und nur eingeschränkt anpassbar ist [4]. Darüber hinaus reduziert sich die Interaktion in der Regel auf rein auditive Ein- und Ausgabe, ohne dass eine physische Präsenz oder visuelle Rückmeldung das Nutzungserlebnis ergänzt. Forschungsarbeiten im Bereich der sozialen Robotik zeigen jedoch, dass die physische Verkörperung eines Systems einen positiven Einfluss auf die wahrgenommene Vertrauenswürdigkeit, die emotionale Bindung und die langfristige Nutzungsbereitschaft haben kann [5], [6]. Ein Assistenzsystem, das als greifbares, sichtbares Gegenüber im Alltag wahrgenommen wird, besitzt demnach ein qualitativ anderes Interaktionspotenzial als ein reiner Softwareagent.

An diesem Punkt setzt das vorliegende Projekt an. Pablo ist ein kompakter Desk-Companion-Bot, der sprachbasierte Assistenzfunktionen mit einer physischen Verkörperung in Form eines 3D-gedruckten Gehäuses, eines integrierten Displays und eines beweglichen Gelenks vereint. Im Unterschied zu bestehenden Systemen verfolgt Pablo einen bewusst modularen und offenen Ansatz: Durch eine Plugin-basierte Softwarearchitektur können Funktionsmodule – etwa Notiz- und Kalenderverwaltung, Timer oder Wetterabfragen – bedarfsgerecht ergänzt und individuell angepasst werden, ohne den Kernprozess verändern zu müssen. Die zugrunde liegende Hardware basiert auf einem Raspberry Pi und ermöglicht eine kostengünstige, eigenständige Umsetzung. Durch die Kombination aus natürlicher Sprachinteraktion, visueller Darstellung und physischer Ausrichtung zur sprechenden Person soll Pablo als persönlicher Companion wahrgenommen werden, der sich nahtlos in den Arbeitsplatz oder das häusliche Umfeld einfügt.

Ziel dieses Projekts ist die Entwicklung eines funktionalen Minimum Viable Products (MVP), das die technische Machbarkeit und das Interaktionskonzept eines solchen Desk-Companion-Bots demonstriert. Die vorliegende Dokumentation beschreibt den vollständigen Entwicklungsprozess – von der Konzeption über die Hard- und Softwareentwicklung bis hin zur Integration und Evaluierung des Gesamtsystems. Dabei orientiert sich die Entwicklung methodisch an der VDI 2221/2206 und gliedert sich in klar definierte Projektphasen.

2 Entwicklungsprozess

Die Entwicklung von Pablo folgte einem iterativen Vorgehen, das sich methodisch an der VDI 2221 /2206 orientiert. Ausgehend von einer initialen Konzeptionsphase wurde das System schrittweise aufgebaut, wobei in jeder Iteration Hardware- und Softwarekomponenten parallel weiterentwickelt und integriert wurden. Im Folgenden werden die Konzeption, die einzelnen Entwicklungsphasen sowie die dabei aufgetretenen praktischen Herausforderungen beschrieben.

2.1 Konzeption

Am Ausgangspunkt der Entwicklung stand die Beobachtung, dass im Alltag häufig kleine Aufgaben, spontane Ideen und Erinnerungen anfallen, die in der Regel unstrukturiert bleiben. Bestehende digitale Assistenzsysteme bieten zwar Unterstützung bei einzelnen Aufgaben, sind jedoch überwiegend als rein softwarebasierte oder in geschlossene Ökosysteme eingebettete Lösungen konzipiert. Die erste Ideenfindung zielte daher auf ein System, das diese Lücke durch eine physisch präsente, offene und erweiterbare Lösung schließt.

Im Rahmen der Zielgruppenanalyse wurde festgestellt, dass insbesondere Personen im häuslichen oder studentischen Umfeld von einem kompakten Schreibtischassistenten profitieren können, der ohne Smartphone oder Computer bedienbar ist. Daraus ergab sich die bewusste Entscheidung, Pablo nicht ausschließlich als funktionalen Assistenten zu konzipieren, sondern als *Companion*, also ein System, das durch seine physische Gestalt, seine visuelle Identität und seine Interaktionsweise eine persönliche Bindung ermöglicht. Die Verkörperung als Pixel-Art-Faultier in einem baumstammförmigen Gehäuse unterstreicht diesen Companion-Charakter und schafft eine niedrigschwellige, sympathische Zugangsmöglichkeit.

Auf Grundlage dieser Überlegungen wurden die Kernfunktionen definiert: Wakeword-basierte Aktivierung, sprachliche Interaktion über ein LLM, konkrete Alltagsfunktionen (Notizen, Kalender, Timer, Wetterabfrage) sowie visuelle Rückmeldung über ein integriertes Display. Diese Funktionen bilden den minimalen Umfang des angestrebten MVP und wurden so gewählt, dass sie den modularen Charakter des Systems demonstrieren.

2.2 Iterative Entwicklung

Die Umsetzung von Pablo erfolgte in fünf aufeinander aufbauenden Entwicklungsphasen, die jeweils mit einem definierten Zwischenstand abgeschlossen wurden.

Phase 1: Grundidee und Funktionsrahmen. In der ersten Phase wurde die grundlegende Systemidee konkretisiert und der funktionale Rahmen festgelegt. Zentrale Fragestellungen waren, welche Rolle Pablo im Alltag der Nutzenden einnehmen soll und welche Minimalfunktionen für einen sinnvollen Prototyp notwendig sind. Ergebnis dieser Phase war ein technisches Konzeptdokument, das die Systemgrenzen, die angestrebte Interaktionsform und die geplante Modulstruktur beschreibt.

Phase 2: Hardwareauswahl und Aufbau. In der zweiten Phase wurden die Hardwarekomponenten des Systems ausgewählt und zu einem ersten physischen Aufbau zusammengeführt. Die Auswahl erfolgte unter Abwägung von Kosten, Kompaktheit, Integrationsfähigkeit und Community-Unterstützung. Im Ergebnis wurde eine Systemarchitektur definiert, die eine zentrale Recheneinheit, eine Mikrofoneinheit, ein Display, einen Lautsprecher mit Verstärker sowie einen Servomotor für die mechanische Ausrichtung des Kopfteils umfasst. Die konkreten Komponenten und deren Zusammenspiel werden in Kapitel **??** beschrieben.

Phase 3: Implementierung der Sprachpipeline. In der dritten Phase wurde die zentrale Verarbeitungskette für die sprachliche Interaktion aufgebaut. Diese umfasst vier sequenzielle Stufen: Wakeword-Erkennung, Spracherkennung (Speech-to-Text), inhaltliche Verarbeitung über ein Large Language Model sowie Sprachausgabe (Text-to-Speech). Ziel dieser Phase war eine erste durchgängig lauffähige Systemkette, in der eine gesprochene Anfrage vollständig erfasst, verarbeitet und beantwortet werden kann. Die einzelnen Komponenten wurden bewusst als austauschbare Module implementiert, um eine spätere Anpassung einzelner Stufen zu ermöglichen.

Phase 4: Integration von Funktionsmodulen. Aufbauend auf der funktionierenden Sprachpipeline wurden in der vierten Phase konkrete Alltagsfunktionen als eigenständige Module integriert. Dazu zählen unter anderem Wetterabfragen, Kalendereinträge, Timer und das Erstellen von Notizen. Die Einbindung erfolgt über eine einheitliche Plugin-Schnittstelle, die es dem System ermöglicht, kontextbezogen zwischen allgemeiner Antwortgenerierung und gezielter Modulausführung zu unterscheiden. Diese Architektur bildet die Grundlage für die angestrebte langfristige Erweiterbarkeit des Systems.

Phase 5: Benutzererlebnis und Persona. In der abschließenden Phase lag der Fokus auf dem Benutzererlebnis und der visuellen Identität von Pablo. Das Gehäuse wurde als 3D-gedruckter Baumstamm gestaltet, in dem sämtliche Hardwarekomponenten untergebracht sind. Auf dem Display wird Pablo als animiertes Pixel-Art-Faultier dargestellt, das je nach Systemzustand unterschiedliche visuelle Zustände zeigt. Darüber hinaus wurde die Interaktion hinsichtlich Antwortzeiten, Sprachausgabequalität und visueller Synchronisation optimiert.

3 Darstellung der Ergebnisse

Test

Im folgenden Abschnitt wird die Umsetzung des entwickelten Prototyps beschrieben. Dabei werden zunächst die funktionalen und nicht-funktionalen Anforderungen dargestellt und danach die darauf basierende Softwarearchitektur sowie das Hardwarekonzept beschrieben.

3.0.1 Funktionale Anforderungen

Das System muss über ein sprachbasiertes Ein- und Ausgabesystem verfügen, das gesprochene Anfragen erfassen und verarbeitete Antworten als Sprache ausgeben kann.

Eine Wakeword-Erkennung ist erforderlich, um das System gezielt aktivieren zu können, ohne dass eine dauerhafte Verarbeitung aller Audioeingaben stattfindet.

Darüber hinaus muss das System eine Erweiterbarkeit bieten, über die zusätzliche Funktionsmodule in das bestehende System integriert werden können.

Auf dem integrierten Display soll ein visueller Avatar dargestellt werden, der den aktuellen Systemzustand widerspiegelt und die Interaktion durch eine grafische Benutzeroberfläche ergänzt.

3.0.2 Nicht-funktionale Anforderungen

Die Bedienung des Systems muss intuitiv gestaltet sein, sodass keine technischen Vorkenntnisse für die Nutzung erforderlich sind.

Die verwendete Hardware muss kompakt dimensioniert sein, damit das Gesamtsystem als Schreibtischgerät eingesetzt werden kann.

Die Integration der einzelnen Hard- und Softwarekomponenten muss robust umgesetzt werden, um einen zuverlässigen Dauerbetrieb zu gewährleisten.

Das System soll echtzeitnahe Reaktionszeiten aufweisen, damit die sprachliche Interaktion als natürlich und flüssig wahrgenommen wird.

Die gesamte Interaktion, einschließlich der sprachlichen Ausgabe, muss verständlich und nachvollziehbar gestaltet sein.

3.1 Softwarearchitektur

Die Softwarearchitektur von Pablo basiert auf einer modularen, ereignisgesteuerten Python-Struktur mit Trennung zwischen Ein- und Ausgabe, Dialogverarbeitung sowie Hardwaresteuerung. Beim Start werden zunächst die Systemkonfiguration, die verfügbaren Plugin-Module und die visuelle Prozessdarstellung initialisiert. Anschließend wird der Wakeword- und Richtungsdetektor konfiguriert, der als Trigger-Komponente für die Interaktionsschleife fungiert. Die aus der Wakeword-Analyse abgeleitete Schallrichtung wird unmittelbar auf die Servosteuerung übertragen, sodass sich der Bildschirm in Richtung der sprechenden Person ausrichtet.

Das System arbeitet mit einer Hauptschleife mit vier logisch getrennten Verarbeitungsphasen: (1) akustische Aktivierung, Richtungsschätzung der Stimme und Anpassung der Servoposition, (2) Spracherkennung mit Speech-to-Text (STT), (3) inhaltliche Verarbeitung über ein Large Language Model (LLM) mit optionaler Modulausführung und (4) Sprachausgabe mit Text-to-Speech (TTS). Die einzelnen Phasen sind funktional entkoppelt und über definierte Funktionsaufrufe miteinander verbunden, wodurch die Austauschbarkeit einzelner Komponenten (z. B. STT- oder TTS-Backend) gewährleistet wird.

Ein zentrales Architekturelement ist die Plugin-Schnittstelle. Über den Loader werden externe Funktionsmodule dynamisch eingebunden und dem LLM-Kontext bereitgestellt. Durch den bereitgestellten Kontext kann das System zwischen generischer Antwortgenerierung und Mo-

dulausführung unterscheiden. Dies ermöglicht eine Erweiterung des Basissystem, ohne dass der Kernprozess verändert werden muss.

Parallelität wird gezielt für zeitkritische Teilaufgaben genutzt. Die Servosteuerung erfolgt asynchron in einem separaten Thread, sodass die Audioverarbeitung nicht blockiert wird. Zusätzlich wird die Zustandsumschaltung der Visualisierung während der Sprachausgabe zeitversetzt und threadbasiert umgesetzt, um eine konsistente Synchronisation zwischen auditivem und visuellem Feedback zu erreichen. Fehlerfälle werden über Ausnahmebehandlung auf Schleifenebene abgefangen.

3.1.1 (1) Aktivierung des Systems

Die Aktivierung erfolgt über eine Wakeword-Erkennung mit Picovoice Porcupine. Beim Erkennen des Wakewords wird zeitgleich die Schallrichtung aus einem gepufferten Stereo-Audiofenster mittels GCC-PHAT-Korrelation der beiden Mikrofonkanäle bestimmt und zur Servo-Ausrichtung genutzt. Die Umsetzung erfolgt in einem integrierten Detector-Modul, das beide Aufgaben zusammenführt.

3.1.2 (2) Spracherkennung (STT)

Die Spracherkennung erfolgt mit dem Vosk-Framework. Der Audiostrom wird verarbeitet, bis das Vosk-Modell einen vollständigen Satz erkannt hat. Erst dann wird der Text an die Dialogverarbeitung übergeben.

3.1.3 (3) LLM-Verarbeitung

Die inhaltliche Verarbeitung nutzt eine API für den Zugriff auf ein Large-Language-Model (Llama 3.3 70B). Der erkannte Nutzereingabe-Text wird zusammen mit den vom Plugin-Loader geladenen Tool-Definitionen an das Modell übergeben. Das Modell entscheidet selbstständig (via `tool_choice=äuto"`), ob ein direkter Chat-Response ausreicht oder ob ein registriertes Plugin-Tool aufgerufen werden soll. Die Plugins kapseln die Logik für Wetter, Notizen, Kalender und Timer.

3.1.4 (4) Sprachausgabe (TTS)

Die Sprachausgabe wird lokal mit Piper generiert (deutsches Modell `de_DE-thorsten-low.onnx`). Der Text wird in eine WAV-Datei gerendert und über einen verfügbaren System-Audio-Player (`paplay` oder `aplay`) abgespielt. Vor dem eigentlichen Audiomaterial wird eine führende Stille eingefügt, sodass die visuelle Feedback-Umschaltung mit dem tatsächlichen Audiobeginn zeitlich synchronisiert ist.

3.2 Hardwarekonzept

Die Auswahl der Hardwarekomponenten erfolgte unter Abwägung von Kosten, Kompaktheit, Integrationsfähigkeit und Community-Unterstützung. Als zentrale Recheneinheit wurde ein Raspberry Pi gewählt, der aufgrund seiner kompakten Bauform, breiten Softwarekompatibilität und um-

fangreichen Community-Dokumentation eine geeignete Plattform für ein Embedded-Assistenzsystem darstellt. Für die Audioerfassung wurde der ReSpeaker 2-Mics Pi HAT eingesetzt, der neben der Sprachaufnahme auch eine richtungsbasierte Schallquellenortung ermöglicht. Die visuelle Ausgabe erfolgt über ein Waveshare 4-Zoll-IPS-Display, das direkt am Raspberry Pi betrieben wird. Als Audioausgabe dient ein 5 W / 8 Ω-Lautsprecher, der über einen separaten Audioverstärker angesteuert wird. Die Stromversorgung des Verstärkers erfolgt über einen Arduino Uno, der zugleich als dedizierte Steuereinheit für die Energieversorgung der Audiokomponente fungiert. Für die mechanische Ausrichtung des Kopfteils, das aus Display und Gehäuseoberteil besteht, wird ein Servomotor eingesetzt, der über einen Arduino Nano angesteuert und seriell vom Raspberry Pi kontrolliert wird. Durch diese Aufteilung der Steuerungsaufgaben auf mehrere Mikrocontroller konnte die Echtzeitfähigkeit der mechanischen und audiovisuellen Komponenten sichergestellt werden, ohne die Hauptrecheneinheit zusätzlich zu belasten.

Das Gehäuse wurde als 3D-gedruckter Baumstamm gestaltet und nimmt sämtliche Hardwarekomponenten auf. Diese Formgebung unterstreicht den Companion-Charakter von Pablo und schafft eine visuelle Abgrenzung gegenüber rein technischen Geräten.

4 Diskussion der Ergebnisse

Die Entwicklung von Pablo zeigt, dass ein kompakter Desk-Companion-Bot mit sprachbasierter Interaktion, visueller Rückmeldung und modular erweiterbarem Funktionsumfang als integriertes System umgesetzt werden kann. Im Vergleich zu rein softwarebasierten Assistenzlösungen wurde mit Pablo bewusst ein physisch präsentes System realisiert, das nicht nur funktionale Aufgaben übernimmt, sondern auch als sichtbarer Begleiter im direkten Nutzungskontext auftritt. Die physische Präsenz, die audiovisuelle Interaktion und die mechanische Ausrichtung des Systems tragen dazu bei, dass die Interaktion stärker als persönlicher Austausch wahrgenommen werden kann.

Die wakeword-basierte Aktivierung stellt dabei eine geeignete Lösung für eine niedrigschwellige und gleichzeitig gezielte Nutzung dar. Das System verbleibt zunächst im Bereitschaftszustand und wird erst nach einer bewussten Ansprache aktiv. In Verbindung mit der Richtungsbestimmung und der Ausrichtung des Displays entsteht ein Interaktionsverhalten, das die Ansprechbarkeit des Systems räumlich sichtbar macht und den Companion-Charakter zusätzlich unterstützt.

Die Kombination aus freier Sprachverarbeitung über ein Large Language Model und strukturierten Funktionsaufrufen erweitert die Einsatzmöglichkeiten des Systems deutlich. Während allgemeine Anfragen flexibel verarbeitet werden können, ermöglichen klar definierte Module die zuverlässige Umsetzung konkreter Alltagsfunktionen. Die implementierten Module für Wetter, Notizen, Kalender und Timer decken typische Anwendungsszenarien im Alltag ab und eignen sich gleichzeitig als prototypische Nachweise für die Erweiterbarkeit der Architektur.

Die modulare Softwarearchitektur erweist sich hierbei als zentrale Grundlage des Systems. Durch die Trennung von Aktivierung, Spracherkennung, Sprachverarbeitung, Sprachausgabe und Hardwaresteuerung konnten einzelne Systembestandteile unabhängig voneinander entwickelt und angepasst werden. Insbesondere die Plugin-Schnittstelle stellt ein wesentliches Ergebnis dar, da sie

eine funktionale Erweiterung des Systems ermöglicht, ohne den Kernprozess grundlegend verändern zu müssen. Dies ist besonders relevant für zukünftige Anpassungen an unterschiedliche Zielgruppen und Nutzungskontexte.

Auch das Hardwarekonzept unterstützt diese Zielsetzung. Die Auswahl kompakter und verbreiteter Komponenten wie Raspberry Pi, ReSpeaker, Waveshare-Display sowie separater Mikrocontroller für Servo- und Verstärkeransteuerung ermöglichte einen integrierbaren und gleichzeitig flexibel entwickelbaren Aufbau. Die Verteilung einzelner Aufgaben auf mehrere Recheneinheiten reduzierte die Belastung der Hauptrecheneinheit und erleichterte die Einbindung mechanischer und audiovisueller Funktionen.

Gleichzeitig zeigt das entwickelte System auch typische Grenzen eines prototypischen Produktes. Die Systemleistung hängt von der zuverlässigen Abstimmung mehrerer Hard- und Softwarekomponenten ab. Insbesondere die Kopplung von Audioverarbeitung, Sprachmodell, visueller Darstellung und mechanischer Bewegung erfordert eine robuste Synchronisation. Darüber hinaus ist die wahrgenommene Natürlichkeit der Interaktion nicht allein vom Sprachmodell abhängig, sondern ebenso von Reaktionszeit, Audioqualität, Wakeword-Zuverlässigkeit und konsistentem visuellen Feedback.

Insgesamt verdeutlichen die Ergebnisse, dass Pablo als modularer Desk-Companion-Bot erfolgreich als funktionaler Prototyp umgesetzt wurde. Besonders hervorzuheben sind die Verbindung aus physischer Präsenz und Sprachinteraktion, die pluginbasierte Erweiterbarkeit sowie die Umsetzung eines kompakten integrierten Hardware-Software-Systems. Damit bildet das entwickelte Produkt eine belastbare Grundlage für weiterführende Entwicklungen und eine gezielte Anpassung an unterschiedliche Anwendungsszenarien.

Im Verlauf der Entwicklung wurden mehrere grundlegende Entscheidungen getroffen, die den Charakter, die Architektur und die technische Umsetzung von Pablo maßgeblich geprägt haben. Im Folgenden werden diese Entscheidungen jeweils mit den betrachteten Alternativen und der jeweiligen Begründung dargestellt.

4.1 Physischer Companion statt reiner Softwareassistent

Eine erste zentrale Entscheidung betraf die Frage, ob Pablo als reine Softwarelösung oder als physisches Produkt umgesetzt werden soll. Softwarebasierte Assistenten lassen sich mit geringerem Aufwand entwickeln und auf bestehenden Endgeräten betreiben. Demgegenüber bietet ein physisches System eine deutlich höhere Präsenz im Alltag der Nutzenden. Forschungsarbeiten im Bereich der sozialen Robotik belegen, dass verkörperte Systeme eine stärkere emotionale Bindung erzeugen und als vertrauenswürdiger wahrgenommen werden als rein virtuelle Agenten. Die Entscheidung fiel daher bewusst zugunsten eines physischen Companion-Bots, um Pablo einen eigenständigen Produktcharakter zu verleihen und eine persönliche Bindung zu ermöglichen, die über die reine Funktionalität hinausgeht.

4.2 Modular erweiterbare Plugin-Struktur

Für die Softwarearchitektur wurde zwischen einer monolithischen Implementierung mit fest integrierten Funktionen und einer modularen Plugin-Struktur mit dynamischer Erweiterbarkeit abgewogen. Eine monolithische Lösung wäre in der initialen Entwicklung einfacher umsetzbar gewesen, hätte jedoch jede funktionale Erweiterung an eine Veränderung des Kernprozesses gebunden. Die Entscheidung fiel auf eine modulare API-basierte Struktur, da diese es ermöglicht, neue Funktionen nachträglich zu ergänzen, ohne bestehenden Code verändern zu müssen. Darüber hinaus erhöht die modulare Architektur die Anpassbarkeit an unterschiedliche Zielgruppen und Nutzungsszenarien und schafft eine Grundlage für ein langfristiges Open-Source-Ökosystem.

4.3 Raspberry Pi als zentrale Recheneinheit

Für die zentrale Recheneinheit wurden verschiedene Plattformen in Betracht gezogen, darunter leistungsfähigere Einplatinencomputer sowie Cloud-basierte Ansätze, bei denen die gesamte Verarbeitung auf einem externen Server erfolgt. Die Entscheidung fiel auf einen Raspberry Pi, da dieser eine gute Verfügbarkeit, eine breite Community-Unterstützung und eine ausreichende Schnittstellenvielfalt für die Anbindung von Mikrofon, Display, Lautsprecher und Mikrocontrollern bietet. Obwohl die Rechenleistung im Vergleich zu leistungsstärkeren Alternativen begrenzt ist, reicht sie für die lokale Koordination der einzelnen Systemkomponenten aus. Zudem eignet sich der Raspberry Pi aufgrund seiner kompakten Bauform besonders gut für den Einsatz in einem Prototyp mit begrenztem Gehäusevolumen.

4.4 Wakeword- und Sprachsteuerung

Für die Aktivierung und Steuerung des Systems wurden neben der Sprachsteuerung auch physische Eingabemethoden wie Tasten oder Touchscreen-Bedienung erwogen. Die Entscheidung fiel auf eine wakeword-basierte Sprachsteuerung, da diese eine niedrigschwellige, freihändige Interaktion ermöglicht. Nutzende können Pablo aktivieren und Befehle erteilen, ohne ihre aktuelle Tätigkeit unterbrechen zu müssen. Dieser Ansatz unterstützt eine möglichst natürliche Zugangsmöglichkeit zum System und entspricht dem Companion-Konzept, bei dem die Interaktion intuitiv und beiläufig erfolgen soll.

4.5 Visuelle Persona

Bei der Gestaltung der Benutzeroberfläche wurde zwischen einer rein funktionalen Darstellung, etwa einer textuellen Statusanzeige, und einer visuellen Persona in Form eines animierten Charakters abgewogen. Die Entscheidung fiel auf die Darstellung eines Pixel-Art-Faultiers als visuelle Identität von Pablo. Diese Gestaltung erzeugt einen hohen Wiedererkennungswert, ermöglicht eine freundlichere und zugänglichere Interaktion und grenzt Pablo bewusst von rein technischen Assistenzsystemen ab. Durch zustandsabhängige Animationen kann zudem der aktuelle Systemstatus auf eine intuitive und ansprechende Weise kommuniziert werden.

4.6 Kompaktes integriertes Display

Alternativ zur Integration eines eigenen Displays wäre eine Ausgabe ausschließlich über Sprache oder über ein externes Gerät wie ein Smartphone möglich gewesen. Die Entscheidung für ein kompaktes, direkt im Gehäuse verbautes Display wurde getroffen, um Pablo ein lokales visuelles Feedback zu ermöglichen. Das Display dient der Darstellung des Avatars, der Anzeige von Statusinformationen und der visuellen Begleitung der Sprachinteraktion. Durch die Integration des Displays in das Gehäuse wird Pablo als eigenständiges, in sich geschlossenes Produkt wahrnehmbar, das keine zusätzlichen externen Geräte für die Nutzung benötigt.

5 Praktische Herausforderungen

Im Verlauf der Entwicklung traten mehrere technische Herausforderungen auf, die exemplarisch für die Komplexität eingebetteter Systeme mit heterogener Hardware- und Softwarelandschaft stehen.

Ein wiederkehrendes Problem betraf den automatischen Systemstart nach dem Booten des Raspberry Pi. Da mehrere Dienste, namentlich die grafische Benutzeroberfläche, der Audiotreiber und der Pablo-Service, beim Hochfahren parallel initialisiert werden, kam es zu Timing-Konflikten. Insbesondere die Audioeingabe über den ReSpeaker-HAT war nach dem Systemstart nicht immer zuverlässig verfügbar, da der entsprechende ALSA-Treiber erst nach einer systemabhängigen Verzögerung vollständig geladen wurde. Diese Problematik wurde durch eine gezielte Startverzögerung und eine Prüfschleife auf Verfügbarkeit der Audioquelle adressiert.

Eine weitere Herausforderung lag in der Integration der unterschiedlichen Hard- und Softwarekomponenten auf der begrenzten Embedded-Plattform. Die Kombination aus Raspberry Pi, zwei Arduino-Mikrocontrollern, dem Mikrofon-HAT, Display und Audioverstärker erforderte eine sorgfältige Abstimmung der seriellen Kommunikation, der Stromversorgung und der GPIO-Belegung. Fehler in der Verkabelung oder Timing-Abweichungen in der seriellen Kommunikation mit den Arduinos führten wiederholt zu schwer reproduzierbaren Fehlerbildern, die nur durch systematisches Testen und schrittweise Integration gelöst werden konnten.

Schließlich stellte die Balance zwischen technischer Funktionalität und einem angenehmen Nutzungserlebnis eine übergreifende Herausforderung dar. So war die reine Funktionsfähigkeit der Sprachpipeline zwar vergleichsweise früh erreicht, die wahrgenommene Qualität der Interaktion wurde jedoch maßgeblich durch Faktoren wie Antwortlatenz, Sprachausgabequalität und die Synchronisation zwischen visueller Darstellung und Audiowiedergabe beeinflusst. Die Optimierung dieser Aspekte erforderte iteratives Anpassen und Testen über mehrere Entwicklungszyklen hinweg.

6 Ergebnis und kritische Einordnung

Pablo ist ein kompakter Desk-Companion-Bot, der zur Unterstützung von Nutzenden im Alltag entwickelt wurde. Das Produkt wurde als physisch präsentes Assistenzsystem konzipiert, das im direkten Umfeld der Nutzenden eingesetzt werden kann und eine niedrigschwellige Interaktion

ermöglicht. Im Unterschied zu rein softwarebasierten Assistenzlösungen basiert Pablo auf dem Ansatz eines physischen Systems, das sprachliche, visuelle und funktionale Elemente in einem Produkt vereint. Ziel des entwickelten Systems ist es, spontane Aufgaben, Fragen und Erinnerungen entgegenzunehmen und in einer möglichst natürlichen Form zu verarbeiten.

Die Interaktion mit dem System erfolgt primär sprachbasiert. Dadurch kann die Nutzung ohne klassische Eingabemedien wie Tastatur, Maus oder Smartphone erfolgen. Ergänzt wird die sprachliche Interaktion durch visuelle Rückmeldungen über ein Display. Durch die Kombination aus physischer Präsenz, visueller Darstellung und sprachlicher Kommunikation wurde ein System entwickelt, das nicht ausschließlich funktionale Assistenzaufgaben erfüllt, sondern zugleich als Companion im persönlichen Umfeld wahrgenommen werden kann.

Ein wesentliches Merkmal von Pablo ist der modulare und personalisierbare Systemaufbau. Die entwickelte Architektur erlaubt es, einzelne Funktionen bedarfsgerecht zu ergänzen. Auf diese Weise kann das Produkt flexibel an unterschiedliche Anforderungen und Nutzungsszenarien angepasst werden. Die Personalisierbarkeit beschränkt sich damit nicht nur auf den Funktionsumfang, sondern betrifft grundsätzlich auch die Ausrichtung des Systems auf verschiedene Anwendungskontexte. Dadurch ist Pablo nicht ausschließlich für ein einzelnes Einsatzszenario ausgelegt, sondern kann perspektivisch für unterschiedliche Zielgruppen eingesetzt werden, beispielsweise als organisatorische Alltagshilfe, als Lernunterstützung oder als individuell konfigurierbarer Companion im privaten Umfeld. Diese Offenheit und Erweiterbarkeit stellt ein zentrales Unterscheidungsmerkmal gegenüber geschlossenen Sprachassistenzsystemen dar, deren Funktionsumfang und Erscheinungsbild in der Regel nur eingeschränkt angepasst werden können.

6.1 Praktische Herausforderungen

Im Verlauf der Entwicklung traten mehrere technische Herausforderungen auf, die exemplarisch für die Komplexität eingebetteter Systeme mit heterogener Hardware- und Softwarelandschaft stehen.

Ein wiederkehrendes Problem betraf den automatischen Systemstart nach dem Booten des Raspberry Pi. Da mehrere Dienste, namentlich die grafische Benutzeroberfläche, der Audiotreiber und der Pablo-Service, beim Hochfahren parallel initialisiert werden, kam es zu Timing-Konflikten. Insbesondere die Audioeingabe über den ReSpeaker-HAT war nach dem Systemstart nicht immer zuverlässig verfügbar, da der entsprechende ALSA-Treiber erst nach einer systemabhängigen Verzögerung vollständig geladen wurde. Diese Problematik wurde durch eine gezielte Startverzögerung und eine Prüfschleife auf Verfügbarkeit der Audioquelle adressiert.

Eine weitere Herausforderung lag in der Integration der unterschiedlichen Hard- und Softwarekomponenten auf der begrenzten Embedded-Plattform. Die Kombination aus Raspberry Pi, zwei Arduino-Mikrocontrollern, dem Mikrofon-HAT, Display und Audioverstärker erforderte eine sorgfältige Abstimmung der seriellen Kommunikation, der Stromversorgung und der GPIO-Belegung. Fehler in der Verkabelung oder Timing-Abweichungen in der seriellen Kommunikation mit den Arduinos führten wiederholt zu schwer reproduzierbaren Fehlerbildern, die nur durch systematisches Testen und schrittweise Integration gelöst werden konnten.

Schließlich stellte die Balance zwischen technischer Funktionalität und einem angenehmen Nutzungserlebnis eine übergreifende Herausforderung dar. So war die reine Funktionsfähigkeit der Sprachpipeline zwar vergleichsweise früh erreicht, die wahrgenommene Qualität der Interaktion wurde jedoch maßgeblich durch Faktoren wie Antwortlatenz, Sprachausgabequalität und die Synchronisation zwischen visueller Darstellung und Audiowiedergabe beeinflusst. Die Optimierung dieser Aspekte erforderte iteratives Anpassen und Testen über mehrere Entwicklungszyklen hinweg.

6.2 Ausblick

Für die Weiterentwicklung des Systems bieten sich mehrere konkrete Ansatzpunkte an. Erstens kann die bestehende Plugin-Architektur um zusätzliche Funktionsmodule erweitert werden, um den praktischen Nutzen in unterschiedlichen Anwendungsszenarien weiter zu erhöhen. Zweitens ist eine stärkere Gehäuse- und Hardwareintegration sinnvoll, um Verkabelung, Wartbarkeit und Langzeitstabilität im Alltagsbetrieb zu verbessern.

Drittens sollte die lokale Ausführung zentraler Komponenten weiter ausgebaut werden, um die Abhängigkeit von externen Diensten zu reduzieren und die Systemrobustheit bei schwankender Netzwerkverfügbarkeit zu erhöhen. Viertens stellt eine vertiefte Personalisierung ein wesentliches Entwicklungspotenzial dar, etwa durch nutzerspezifische Dialogprofile, adaptive Interaktionsmuster und konfigurierbare Verhaltensweisen. Insgesamt bildet der aktuelle Prototyp damit eine tragfähige Basis für den Übergang von einer funktionsfähigen Demonstrationsplattform zu einem langfristig einsetzbaren Companion-System.

Literatur

[1] M. B. Hoy, „Alexa, Siri, Cortana, and More: An Introduction to Voice Assistants," *Medical Reference Services Quarterly*, Jg. 37, Nr. 1, S. 81–88, 2018. DOI: 10.1080/02763869.2018.1404391

[2] V. Kepuska und G. Bohouta, „Next-Generation of Virtual Personal Assistants (Microsoft Cortana, Apple Siri, Amazon Alexa and Google Home)," *IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*, S. 99–103, 2018. DOI: 10.1109/CCWC.2018.8301638

[3] W. X. Zhao u. a., „A Survey of Large Language Models," *arXiv preprint arXiv:2303.18223*, 2023.

[4] F. Bentley, C. Luvogt, M. Silverman, R. Wirasinghe, B. White und D. Lottridge, „Understanding the Long-Term Use of Smart Speaker Assistants," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, Jg. 2, Nr. 3, S. 1–24, 2018. DOI: 10.1145/3264901

[5] J. Li, „The Benefit of Being Physically Present: A Survey of Experimental Works Comparing Copresent Robots, Telepresent Robots and Virtual Agents," *International Journal of Human-Computer Studies*, Jg. 77, S. 23–37, 2015. DOI: 10.1016/j.ijhcs.2015.01.001

[6] T. Belpaeme, J. Kennedy, A. Ramachandran, B. Scassellati und F. Tanaka, „Social Robots for Education: A Review,“ *Science Robotics*, Jg. 3, Nr. 21, eaat5954, 2018. DOI: 10.1126/scirobotics.aat5954